VoiceOps 2026

Building a More Natural Voice-Support Experience for ZentroMart

1. Introduction

Customer support is often the first place where people notice how easy—or frustrating—it is to use an online service. A customer may simply want to know when an order will arrive, whether an item can be returned, or how to get help with a problem. Making customers type every question can be inconvenient, especially when they would rather explain it naturally by speaking. VoiceOps explores a practical alternative: a voice-enabled support assistant that listens to a customer's question, uses the store's knowledge base to find relevant information, and responds both in text and spoken audio.

The project is built around ZentroMart, a fictional online retailer used in the supplied capstone materials. The aim is not to replace every human support interaction. Instead, the prototype demonstrates how common questions can be handled more smoothly, while verification and support-ticket flows can help route cases that need additional care.

2. Project objectives

The project brings several components together into one end-to-end workflow:

- **Listen:** accept a customer audio recording and convert speech into text using Gnani speech-to-text (STT).
- **Understand and retrieve:** search the provided ZentroMart policy documents for information relevant to the customer's question.
- **Respond:** use a Groq-hosted language model to generate a clear answer grounded in the retrieved evidence and the conversation context.
- **Speak:** turn the written response into audio using Gnani text-to-speech (TTS).
- **Keep a record:** save transcripts, replies, retrieved evidence, and output-audio paths in JSON so a run can be inspected afterward.

3. How the system works

Step	What happens
1. Customer audio	A supported audio file is provided as input. The CLI accepts WAV, MP3, M4A, and FLAC files.
2. Speech-to-text	Gnani STT transcribes the customer's speech. For the tested Hindi example, an MP4 recording was first converted to W
3. Policy retrieval	The query is embedded with a multilingual sentence-transformer model and compared with embedded policy chunks.
4. Response generation	The retrieved excerpts and conversation context are passed to the Groq-hosted model to produce a customer-facing re
5. Text-to-speech	Gnani TTS synthesizes the reply into a WAV file, making it possible to hear the answer rather than only read it.
6. Output logging	The run saves per-input JSON and a combined results file, including the transcript, response, audio path, and retrieved

4. Knowledge-base grounding and return-policy handling

A voice assistant is only useful when its answer reflects the policy that actually applies. During testing, a Hindi question asked how many days a customer has to return clothing. The first retrieval result overemphasized a special festive-sale rule, making the normal return window less clear. Reviewing the knowledge-base document showed two distinct rules: the standard return window in RET-2 and the festive-sale exception in RET-8.

The standard policy states that clothing, footwear, and bags can normally be returned within 15 calendar days from delivery. It also states that qualifying orders placed between 1 October and 15 October 2026 can be returned within 30 days of delivery. That longer window is an exception for qualifying orders, not a replacement for the standard rule.

To reduce the chance of the exception crowding out the standard rule, the retrieval logic in *src/agent.py* was updated to include the authoritative RET-2 section when a return-related query is detected. This is a targeted

safeguard for the observed failure case; it should not be interpreted as proof that every policy question is now solved. Broader tests are still needed across categories, languages, and edge cases.

5. Testing and observed results

The working notebook was tested in Google Colab. The test used a genuine Hindi customer-question recording stored in an MP4 file; its audio was extracted to WAV before passing it through the STT stage. Gnani STT produced the Hindi question about the clothing return window. The agent's first answer did not clearly prioritize the standard rule, which exposed a weakness in retrieval ranking and chunk boundaries.

After applying the targeted retrieval adjustment in the notebook, the agent answered that clothing, footwear, and bags normally have a 15-day return window from delivery, and then conditionally mentioned the 30-day festive-sale exception. Gnani TTS subsequently generated a WAV response that was played successfully. These checks demonstrate a successful smoke test of this specific path in the existing runtime.

The tests described above are deliberately reported as a smoke test, not a complete evaluation. A clean installation from scratch, a broad set of audio files, and a systematic pass over all customer-support scenarios have not been established by these results.

6. Main project components

Component	Responsibility
VoiceOps.ipynb	Development notebook used for setup, experiments, and interactive tests.
src/agent.py	Knowledge-base loading, embeddings, retrieval, model interaction, and support conversation logic.
run.py	Command-line batch workflow that processes audio files and writes JSON and audio outputs.
requirements.txt	Python package dependencies for the project.
.env.example	Lists environment variables expected for local configuration without containing real credentials.
README.md	Setup instructions, dataset expectations, CLI usage, and project limitations.

7. Running the project

A local run requires the capstone dataset, Python dependencies, and valid API credentials. Extract the supplied dataset and set `VOICEOPS_DATA_DIR` to the extracted `Capstone_Project_Dataset_FTTT_2026` folder. The folder is expected to contain the `data/`, `kb/`, and `tools/` directories.

Install dependencies with `pip install -r requirements.txt`. Copy `.env.example` to a local `.env` and add the required credentials there. Do not push the local environment file to GitHub.

Example command:

```
python run.py --input_dir path/to/audio --output_dir sample_outputs --language hi-IN
```

The command-line runner accepts WAV, MP3, M4A, and FLAC inputs. It writes JSON results and generated reply audio into the output directory. MP4 is not currently accepted directly by the runner, so its audio must first be extracted to a supported format.

8. Limitations and future improvements

The prototype is a useful integration demonstration, but several improvements would make it more robust:

- Improve chunking so policy headings, table rows, and exceptions stay together in the same retrieval unit.
- Add a repeatable test set for each return category, festive-sale eligibility, refunds, cancellations, order lookup, failed verification, and ticket creation.
- Evaluate Hindi and English inputs separately, including noisy audio, partial transcripts, and alternate phrasings.
- Make tool outcomes explicit: only report an order lookup or ticket creation as successful when the relevant tool confirms it.
- Run the complete project from a fresh environment to confirm that setup instructions and dependencies are sufficient.

- Add a short screen-recorded demonstration and representative, non-sensitive sample outputs to the repository.

9. Security and responsible handling

API credentials should remain in a local environment file or approved secret store. The dataset archive, private customer recordings, generated audio containing personal information, virtual environments, and cache folders should not be committed to the public repository. When testing order-related flows, use the supplied test data and follow the agent's verification process before exposing order details.

10. Conclusion

VoiceOps brings speech recognition, knowledge-base retrieval, language-model response generation, and speech synthesis into one customer-support workflow. The tested Hindi return-policy scenario showed both the value of grounding answers in the source documents and the risk of retrieving a special-case policy without the standard rule. A targeted safeguard has been added to the repository to address that observed issue. The next step is to validate the change from a clean setup and broaden the test coverage before treating the prototype as submission-ready.